__/__/__

Week-7 DL Gen AI

L1 Introduc^{n} to Sequence Modeling

- RNNs help to model sequences.
- CNN helps in vision by paramter sharing and sparse connections.
- RNNs were used in language modelling ⇒ slowly going into Gen AI.
  eg. predict stn. in future, st. there was given some prior information.
- seq. data → kind of data where the sequence of things actually matter. The seq. contains impt. info.
  → each data pt. is dependent on previous ones. (no i.i.d) ⇒ temporal / spatial dependency
- Key properties —
  1. ordering ⇒ tokens in a seq. (like I word)
  2. vary. length → seq. may have diff. len.
  3. dependencies ⇒ complex dependency at diff. posi^{n} within the sequence.
- Idea → analyze data to understand patterns, trends over time or space. (vision)
- some common tasks → predictive maintainence, speech recogn^{n}, music gen., sentiment classifica^{n}, DNA sequence analysis, machine transla^{n}, video activity recogni^{n}, NER (named entity recog^{n})
  ⇒ remember, everything gets converted to no., because systems only understand numbers.

L2 **Mathematical Modeling of Sequences**

- Seq. model aims to estimate the joint probability of an entire sequence of observations.

$x_1, x_2 \dots x_T$, the model estimates

$$P(x_1, x_2, \dots, x_T)$$

⇒ given a seq. of words, estimate its probab., which ultimately helps in generation as well.

- When a seq. model is applied to a text data, it is called as LM (language model). The goal is to estimate the probability of a sequence of words or characters.
- Autoregressive Model (GPTs) ⇒ predict future values in a seq. based on past values. Uses chain rule.

$$P(x_1, x_2 \dots x_T) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_1, \dots, x_{t-1})$$

↳ probab. of a seq. is the product of the conditional probabilities of each element, given the elements that came before it.

x_t $\xrightarrow{\text{depends on}}$ $(x_1, \dots, x_{t-1})$ as input.

- Handling very long sequences are computationally expensive because model will have to learn entire history

↳ solution ⇒ Markov chain

$$P(x_1, \dots, x_T) \approx P(x_1) \prod_{t=2}^{T} P(x_t \mid x_{t-1})$$

↳ bigram model in lang. processing.

we assume that the current state x_t only depends on immediately preceding state x_{t-1}. ⇒ we can have fixed-len window of prev. states.

- LR for Seq. modelling
  → n-gram markov assump^n
  → fixed window of past data & applies a linear model to predict the next value.

$$\hat{x}_t = \sum_{i=1}^{\tau} w_i x_{t-i} + b$$

  - w_i: model wts. to be learned
  - x_{t-i}: past values
  - b: bias term

→ problems
1. linearity assump^n → just a weighted sum. They do not capture complex relations
2. lack of long-term memory (fixed window) only fixed (τ).
3. error accumula^n → predic^{ns} might diverge rapidly from the true data.

- why MLPs & CNNs are not ideal for sequences
1. fixed-size vectors input ⇒ truncate long sequences.
2. no parameter-sharing across time.
3. no memory of context

- now we need a model that processes sequences step-by-step, with memory.

L3 **Language Modelling**

- pose the language problem as a sequencing task.
- first, make text understandable to machine
  convert raw text corpus ⟹ numerical format which a model can process.

1. Read Data → 2. Tokenize create tokens → 3. Numerical Seq. Build the vocabulary

- vocab / dictionary maps every unique word or character to a unique integer index.
- 1st step was splitting the text into tokens.
  1. Character Tokens → indi. charac.
     Small vocab., but long sequences.
  2. Word Tokens → split by spaces & punctuation.
     More common, large vocab, but shorter meaningful sequences.
- how to build V → find all unique tokens in a text corpus & then assign a unique integer index to each one. Then, we simply replace each token with its corresponding index from the vocab.
- LM has to find the probab. of given seq.

$$P(x_1 \dots x_T) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_1, \dots, x_{t-1})$$

probab. of a word depends on all the words before it.

- the challenge is $P(x_t | x_1, \dots x_{t-1})$ becomes computationally expensive. This is the major problem of statistical models.
- To train a neural LM, we need feature-label pairs. 2 strategies — Random sampling and Sequential Partitioning.
  ↳ idea → next token prediction.
- Random sampling
  1. random tokens are discarded from the start of corpus in each epoch.
  2. Rest is partitioned into non-overlapping seq. of fixed len L.
  3. $X = (x_1, x_2 \dots L)$. Y is shifted by 1 token' $\Rightarrow Y = (x_2, x_3, \dots x_{L+1})$
  4. Now each subsequence is treated as independent - eg. of training.

Now,
no. of x-y pairs = no. of data pts. for training.

- Perplexity (PPL) → std. metric to evaluate the lang. model. How surprised a model by the seq. of text.
  → lower the PPL, better the performance.
  → it runs on test set & not train set.

$$PPL = \exp\left(-\frac{1}{T}\sum_{t=1}^{T} \log P(x_t | x_1, \dots x_{t-1})\right)$$

T: text len; CE loss

__/__/____

$$PPL = \frac{1}{\underbrace{\prod_{t=1}^{T} P(x_i | x_{-i-1})^{1/T}}_{\text{probability of sequence}}}$$

→ PPL of k means that on avg., the model is as uncertain about the next word as if it had to choose uniformly from k diff. words.

at least your PPL < v.

L4. Recurrent Neural Networks

→ general sequence models. It can handle variable length data. Used to predict the next term in the sequence.

- Makov & n-gram models have fixed memory. They look at only (n-1) tokens. To remember more, the no. of params grow exponentially.
- RNN, model approximates the term probab. with the hidden state.

$$P(x_t | x_1, \ldots, x_{t-1}) \approx P(x_t | h_{t-1})$$

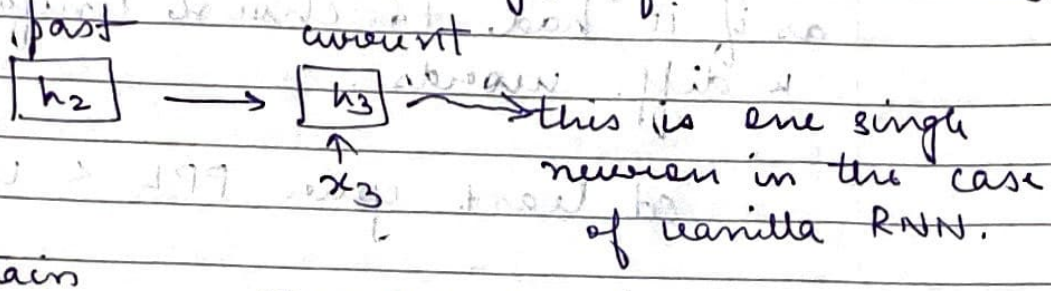
$$h_t = f(x_t, h_{t-1})$$

↙ hidden state, zipped form of the sequence till now. ⇒ kind of a summary.

- RNN processes a seq. by iterating through its elements one by one. The loca^n of y is optimal Obv., the more the context, the answer will be better.

__/__/__

- It maintains a hidden state (or memory) that captures info. abt. what it has seen so far. At each time step, the network -
  1. takes a new input x_t
  2. updates its hidden state h_t
  3. produces an output y_t (if we desire so)

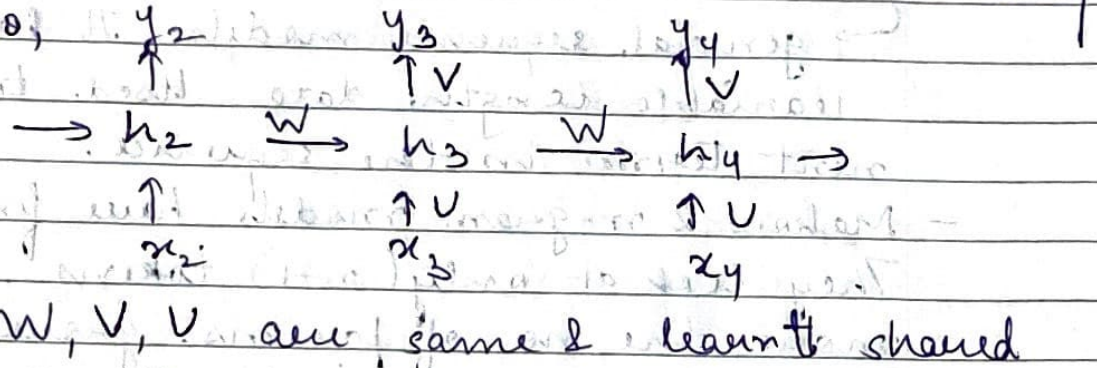


maintain the sizes of h_3.

$$z_3 = U x_3 + W h_2$$
$$h_3 = g(z_3)$$

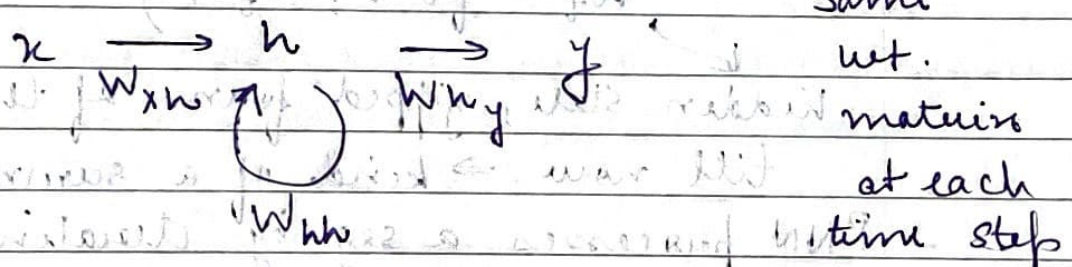
↳ some non-linearity

Also,



W, V, U are same & learnt shared parameters.

- A recurrent neuron (fold it)



There can be many such recurrent neuron forming a layer.

- for 1 neuron.

$$h_{(t)} = \tanh\left(W_{xh} x_{(t)} + W_{hh} h_{(t-1)} + b_h\right)$$

$$y_t = W_{hy} h_{(t)} + b_y$$

the dimension of y depends entirely on the task.

Eg. $v = \{ h:0, e:1, l:2, o:3 \}$

input is e, memory has seen h, predict next character ⇒ ?

$x_t('e') : (0\ 1\ 0\ 0)^T$
$y_t('l') : (0\ 0\ 1\ 0)^T$
} 2 hidden neurons we are using one hot enc.

$h_{t-1}('h') : \begin{pmatrix} 0.6 \\ 0.2 \end{pmatrix}$

$W_{xh} = \begin{pmatrix} 0.1 & 0.4 \\ 0.2 & 0.5 \\ 0.3 & 0.6 \\ 0.4 & 0.7 \end{pmatrix}$ $W_{hh} = \ldots$ $W_{hy} = \ldots$

$b_h = \ldots$ $b_y = \ldots$

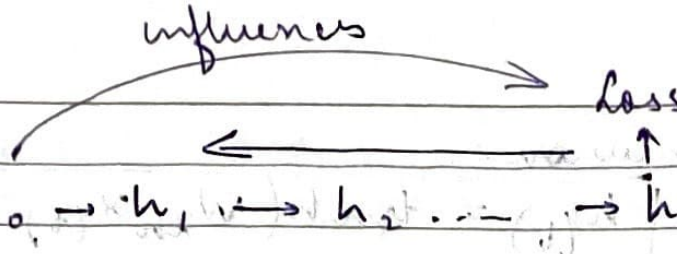
then calculate using the above formula.

$h_t = \begin{bmatrix} 0.43 \\ 0.72 \end{bmatrix}$ → encodes h seen now.

$y_t = \begin{bmatrix} 0.24 \\ 0.63 \\ 2.89 \\ 0.61 \end{bmatrix} \begin{matrix} \rightarrow h \\ \rightarrow e \\ \rightarrow l \\ \rightarrow o \end{matrix}$

generally softmax is applied to get the probability.

- backpropaga" has some challenges →
→ dependencies thru time.



- This ↓d BPTT (Backpropagation through time). It is std. algo. to the unrolled computational graph of RNN.
- For a seq. of len T, calculating the gradient for the initial vals involves a chain rule product of T matrices. This long product is the source of major numerical instability

  Eigen values $> 1 \Rightarrow$ exploding gradients
  " $< 1 \Rightarrow$ vanishing gradients

- Gradient Explosion
  ↓ soln
  gradient clipping → if the norm of the gradient exceeds a certain threshold, it is scaled down.
  (the direc" is still maintained)

- Gradient Vanishing
  ↓ soln
  → more complex RNN architecture
  → truncated BPTT → so you backpropagate over only a limited no. of step, thus keeping the memory small.
  ↳ though, it limits the model's ability to learn dependencies that span longer than T time steps

__/__/____

L5 Vanilla RNN - Python demo

- yFinance → popular Python lib. for fetching historical market data from Yahoo Finance.
- collect the data for 'AMZN', period, interval
- extract the relevant columns for us.
- plot the data to understand trend.
- define the lookback period. How much data it is allowed to check?
- convert df to numpy (more efficient storage & computational format)
- split the train & test dataset
  ↳ convert it into a time-series dataset ⇒ set it into dataloader.
- convert the dataset into tensor object before going into the dataloader.
- perform the forward pass, accumulate loss, zero out the gradients, do the backprop, and update your weights.
- for inference, no gradients should be configured.
- use the trained model for test dataset.